

Divisão e conquista

Multiplicação de inteiros grandes (Karatsuba)

Prof. Dr. André Yoshiaki Kashiwabara

Departamento de Computação — Universidade Federal de Sergipe

6 de outubro de 2026

1. O problema: multiplicar inteiros de n dígitos em $\Theta(n^2)$



2. Divisão ingênua: 4 subproblemas, nenhum ganho



3. Karatsuba: 3 subproblemas e $\Theta(n^{1,585})$



4. Na prática e na história: corte, Toom, FFT

trocar um produto
por somas

Na próxima aula, a mesma ideia aplicada a matrizes: Strassen.

O problema

Problema:

Um `long long` guarda só 19 dígitos. Muita coisa precisa de mais:

- **criptografia:** uma chave RSA de 2048 bits tem cerca de **617 dígitos** decimais, e cifrar exige muitas multiplicações;
- **linguagens:** o `int` do Python não tem limite de tamanho;
- **álgebra computacional:** produtos exatos de polinômios e frações.

Solução?

O algoritmo da escola faz n^2 multiplicações de dígitos. Para $n = 10^6$ dígitos são 10^{12} produtos: uns 17 minutos a 10^9 por segundo.

Pergunta da aula: dá para fazer com *menos* que n^2 ?
Em 1960, Kolmogorov conjecturou que não.

Para aprofundar: Levitin (2012), §5.4 [10]; Knuth (1997), §4.3.3 [9]; Karatsuba (1995) [7].

Definição

Um inteiro de n dígitos na base B (aqui $B = 10$) é

$$x = \sum_{i=0}^{n-1} x_i B^i, \quad 0 \leq x_i < B.$$

Guardamos os dígitos num vetor, do *menos* significativo (x_0) para o mais significativo, como um polinômio avaliado em B .

Intuição

No produto da escola, cada dígito de y multiplica todos os n dígitos de x : são $n \cdot n = n^2$ produtos de um dígito, mais as somas com vai-um.

Para aprofundar: Knuth (1997), §4.3.1, Algoritmo M [9]; Levitin (2012), §5.4 [10].

$$\begin{array}{r} 2135 \\ \times 4014 \\ \hline 8540 \\ 2135 \\ 0000 \\ + 8540 \\ \hline 8569890 \end{array}$$

$4 \times 4 = 16$ produtos de um dígito

```
/* r = x y; x[0] é o dígito menos significativo;
   r tem 2n posições, todas zeradas */
void multEscola( int n, const int *x,
                 const int *y, int *r) {
    for (int j = 0; j < n; ++j) {
        int vaiUm = 0;
        for (int i = 0; i < n; ++i) {
            int t = r[i+j] + x[i] * y[j] + vaiUm;
            r[i+j] = t % 10;
            vaiUm = t / 10;
        }
        r[j+n] = vaiUm;
    }
}
```

Contagem:

- o laço interno roda $n \cdot n$ vezes;
- cada volta faz 1 produto de dígitos e $O(1)$ somas;
- total: $\Theta(n^2)$.

Dobrar n multiplica o tempo por $2^2 = 4$ (medido no tutorial).

Somar dois números de n dígitos custa só $\Theta(n)$. Essa diferença é o que Karatsuba vai explorar.

Para aprofundar: Knuth (1997), §4.3.1, Algoritmo M [9].

O que aprendemos

- Inteiros grandes são vetores de dígitos: $x = \sum x_i B^i$.
- Somar custa $\Theta(n)$; multiplicar pelo método da escola custa $\Theta(n^2)$.
- Multiplicar por B^m é só deslocar m casas (acrescentar zeros), com custo $\Theta(n)$ e nenhuma multiplicação de dígitos.
- Ler a entrada já custa $\Theta(n)$. A pergunta é onde fica o custo entre n e n^2 .

Para aprofundar: Knuth (1997), §4.3.1 e §4.3.3 [9].

Divisão ingênua

Problema:

Mergesort e o par mais próximo nos ensinaram a quebrar a entrada ao meio. Um número de n dígitos se parte naturalmente em duas **metades** de $m = n/2$ dígitos.

Solução?

Escrever $x = a \cdot 10^m + b$ e $y = c \cdot 10^m + d$ e multiplicar as metades, recursivamente.

$$\begin{array}{l} X = \overbrace{\begin{array}{|c|c|} \hline 2 & 1 \\ \hline \end{array}}^{a = 21} \overbrace{\begin{array}{|c|c|} \hline 3 & 5 \\ \hline \end{array}}^{b = 35} \\ y = \overbrace{\begin{array}{|c|c|} \hline 4 & 0 \\ \hline \end{array}}^{c = 40} \overbrace{\begin{array}{|c|c|} \hline 1 & 4 \\ \hline \end{array}}^{d = 14} \end{array}$$

$$2135 = 21 \cdot 10^2 + 35, \quad 4014 = 40 \cdot 10^2 + 14$$

Para aprofundar: Kleinberg e Tardos (2006), §5.5 [8]; Levitin (2012), §5.4 [10].

Produto pelas metades

$$x y = (a \cdot 10^m + b)(c \cdot 10^m + d) = ac \cdot 10^{2m} + (ad + bc) \cdot 10^m + bd.$$

No exemplo: 2135×4014

$$ac = 21 \cdot 40 = 840, \quad ad = 21 \cdot 14 = 294, \quad bc = 35 \cdot 40 = 1400, \quad bd = 35 \cdot 14 = 490.$$

$$840 \cdot 10^4 + (294 + 1400) \cdot 10^2 + 490 = 8\,400\,000 + 169\,400 + 490 = 8\,569\,890 \checkmark$$

Custo: 4 produtos de $n/2$ dígitos (recursivos), mais deslocamentos e 3 somas, todos $\Theta(n)$.

Para aprofundar: Kleinberg e Tardos (2006), §5.5 [8]; Levitin (2012), §5.4 [10].

$$T(n) = 4 T(n/2) + \Theta(n)$$

$a = 4$, $b = 2$, $d = 1$: como $4 > 2^1$, as **folhas dominam**:

$$T(n) \in \Theta(n^{\log_2 4}) = \Theta(n^2).$$

Na árvore, o nível j tem 4^j nós de custo $n/2^j$:

$$4^j \cdot \frac{n}{2^j} = n 2^j,$$

uma série que *dobra* a cada nível. As n^2 folhas fazem quase todo o trabalho.

Para aprofundar: Cormen et al. (2009), §4.5, Teorema 4.1 [1]; Levitin (2012), §5.4 [10].



Lição: como as folhas dominam, só adianta reduzir a , o *número* de subproblemas.

O que aprendemos

- $x = a \cdot 10^m + b$ e $y = c \cdot 10^m + d$ dão $xy = ac \cdot 10^{2m} + (ad + bc) \cdot 10^m + bd$.
- 4 produtos de $n/2$ dígitos e somas $\Theta(n)$: $T(n) = 4T(n/2) + \Theta(n) = \Theta(n^2)$.
- Dividir por dividir não basta. Quando as folhas dominam, baratear a combinação não adianta; é preciso **cortar subproblemas**.

Para aprofundar: Kleinberg e Tardos (2006), §5.5 [8].

O algoritmo de Karatsuba

Problema:

Não precisamos de ad e bc separados: só da soma $ad + bc$. Gastar dois produtos com ela é desperdício.

Solução:

um único produto já contém tudo:

$$(a + b)(c + d) = ac + (ad + bc) + bd.$$

Como ac e bd já serão calculados, basta subtraí-los:

$$ad + bc = (a + b)(c + d) - ac - bd.$$

O mesmo truque, 150 anos antes

Gauss multiplicava complexos com 3 produtos reais:

$$(a + bi)(c + di) = (ac - bd) + (ad + bc)i,$$

com $ad + bc = (a + b)(c + d) - ac - bd$.

	produtos	somas/subtr.
ingênuo	4	3
Karatsuba	3	6

Para aprofundar: Dasgupta et al. (2008), §2.1 [3]; Karatsuba e Ofman (1962) [6].

Algoritmo de Karatsuba (1962)

Com $x = a \cdot 10^m + b$ e $y = c \cdot 10^m + d$, calcule recursivamente

$$z_2 = ac, \quad z_0 = bd, \quad z_1 = (a + b)(c + d) - z_2 - z_0,$$

e devolva

$$xy = z_2 \cdot 10^{2m} + z_1 \cdot 10^m + z_0.$$

Intuição

Trocamos uma multiplicação (cara, recursiva) por três somas/subtrações (baratas, $\Theta(n)$). O ganho de um produto em quatro se repete em *cada nível* da recursão.

Detalhe: $a + b$ pode ter $m + 1$ dígitos (vai-um). Isso não muda a análise.

Para aprofundar: Karatsuba e Ofman (1962) [6]; Levitin (2012), §5.4 [10]; Kleinberg e Tardos (2006), §5.5 [8].

Exemplo: 2135×4014 por Karatsuba



$m = 2$: $a = 21, b = 35, c = 40, d = 14$.

$$z_2 = 21 \cdot 40 = 840$$

$$z_0 = 35 \cdot 14 = 490$$

$$(a + b)(c + d) = 56 \cdot 54 = 3024$$

$$z_1 = 3024 - 840 - 490 = 1694$$

Confira: $ad + bc = 294 + 1400 = 1694 \checkmark$

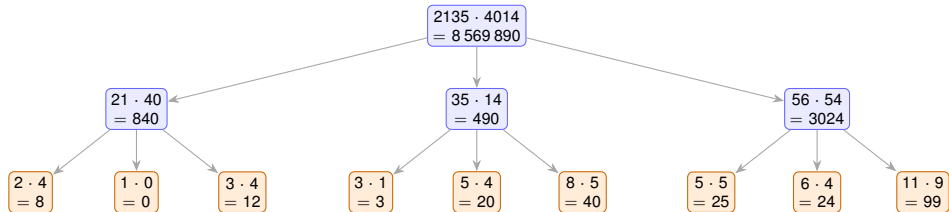
Combinação

$$\begin{aligned} x y &= 840 \cdot 10^4 + 1694 \cdot 10^2 + 490 \\ &= 8\,400\,000 + 169\,400 + 490 \\ &= 8\,569\,890 \checkmark \end{aligned}$$

Foram **3** produtos de dois dígitos em vez de 4. E cada um deles também é feito por Karatsuba (próximo slide).

Para aprofundar: Levitin (2012), §5.4, exemplo $2135 \cdot 4014$ [10].

A árvore de recursão do exemplo



Em cada nó: $z_2, z_0, (a + b)(c + d)$. Ex.: $21 \cdot 40$ tem $z_2 = 8, z_0 = 0, z_1 = 12 - 8 - 0 = 4$, logo $8 \cdot 100 + 4 \cdot 10 + 0 = 840$. **9 produtos** de um dígito (3^2), contra 16 (4^2) da escola. O $11 \cdot 9$ vem do vai-um em $5 + 6$.

Para aprofundar: Levitin (2012), §5.4 [10].

```
KARATSUBA(x, y, n):           /* n dígitos */
  se n <= CORTE: devolva ESCOLA(x, y)
  m = n/2
  a, b = x div 10^m, x mod 10^m
  c, d = y div 10^m, y mod 10^m
  z2 = KARATSUBA(a, c, m)
  z0 = KARATSUBA(b, d, m)
  p = KARATSUBA(a+b, c+d, m+1)
  z1 = p - z2 - z0
  devolva z2*10^(2m) + z1*10^m + z0
```

Dividir: separar as metades e formar $a + b$, $c + d$: $\Theta(n)$.

Conquistar: 3 chamadas de tamanho $\approx n/2$.

Combinar: 2 subtrações, deslocamentos e somas: $\Theta(n)$.

“ $\cdot 10^m$ ” é só deslocar dígitos; “div” e “mod” por 10^m são só escolher metades do vetor. Nada disso multiplica. A versão em C está no tutorial.

Para aprofundar: Kleinberg e Tardos (2006), §5.5, RECURSIVE-MULTIPLY [8]; Dasgupta et al. (2008), §2.1 [3].

$$T(n) = 3 T(n/2) + \Theta(n)$$

$a = 3, b = 2, d = 1$: $3 > 2$, as folhas dominam:

$$T(n) \in \Theta(n^{\log_2 3}) \approx \Theta(n^{1,585}).$$

Contando só produtos de dígitos, com $M(1) = 1$:

$$M(n) = 3 M(n/2) \Rightarrow M(2^k) = 3^k = n^{\log_2 3}.$$

O nível j custa $3^j \cdot \frac{n}{2^j} = n (3/2)^j$: ainda cresce, mas mais devagar.

n	escola 4^k	Karatsuba 3^k	razão
$2^4 = 16$	256	81	3,2
$2^7 = 128$	$1,6 \times 10^4$	$2,2 \times 10^3$	7,5
$2^{10} = 1024$	$1,0 \times 10^6$	$5,9 \times 10^4$	18
$2^{20} \approx 10^6$	$1,1 \times 10^{12}$	$3,5 \times 10^9$	315

A razão é $(4/3)^k$. Com um milhão de dígitos, os 17 minutos caem para poucos segundos.

Para aprofundar: Levitin (2012), §5.4 [10]; Cormen et al. (2009), §4.5 e Problema 30-1 [1].

O que aprendemos

- $ad + bc = (a + b)(c + d) - ac - bd$: 3 produtos em vez de 4, pagando com somas.
- $T(n) = 3T(n/2) + \Theta(n) \in \Theta(n^{\log_2 3}) \approx \Theta(n^{1,585})$: o *expoente* cai, e não só a constante.
- No exemplo 2135×4014 : 9 produtos de um dígito em vez de 16.
- As fórmulas valem para polinômios (troque 10^m por t^m), por isso as bibliotecas usam Karatsuba também para multiplicar polinômios.

Para aprofundar: Karatsuba e Ofman (1962) [6]; Cormen et al. (2009), Problema 30-1 [1].

Na prática e na história

Problema:

Em números pequenos, as somas extras e as chamadas recursivas custam mais do que o produto que economizam.

Contando operações da implementação do tutorial: a escola faz $2n^2$; um nível de Karatsuba faz $3 \cdot 2(n/2)^2 + 4n - 3 = 1,5n^2 + 4n - 3$. Compensa só se

$$0,5n^2 > 4n - 3 \iff n \geq 8.$$

Solução: um ponto de corte.

Abaixo dele, usa-se a escola.

- **CPython:** `KARATSUBA_CUTOFF = 70` “dígitos” de 30 bits (cerca de 630 dígitos decimais);
- **GMP:** o limiar é medido em cada máquina por um programa de ajuste e, acima de outros limiares, troca Karatsuba por Toom-3, Toom-4, ..., FFT.

Na prática, o corte medido é maior que o da contagem: chamadas e memória também custam.

Para aprofundar: CPython, `Objects/longobject.c` [11]; GMP, “Multiplication Algorithms” [4].

1960, Universidade de Moscou. Kolmogorov abre um seminário sobre complexidade e conjectura que multiplicar dois números de n dígitos exige $\Omega(n^2)$ operações.

Em cerca de uma semana, **Anatolii Karatsuba**, aluno de 23 anos, encontra o método de $n^{\log_2 3}$. Kolmogorov apresenta o resultado na sessão seguinte e encerra o seminário.

Em 1962 sai o artigo assinado por Karatsuba e Ofman, escrito por Kolmogorov. Karatsuba só soube do artigo ao receber as separatas.

Lição

Uma cota inferior “óbvia” precisa de prova. O método da escola era usado havia séculos, e ninguém tinha provado que ele era ótimo, porque ele não é.

Para aprofundar: Karatsuba (1995) [7]; Knuth (1997), §4.3.3 [9].

algoritmo	ano	ideia	custo
escola	—	n^2 produtos	$\Theta(n^2)$
Karatsuba	1962	2 partes, 3 produtos	$n^{1,585}$
Toom-3	1963	3 partes, 5 produtos	$n^{1,465}$
Toom- k		k partes, $2k - 1$ produtos	$n^{\log_k(2k-1)}$
Schönhage–Strassen	1971	FFT	$n \log n \log \log n$
Harvey–van der Hoeven	2019	FFT multidimensional	$n \log n$

Toom- k generaliza Karatsuba: corta em k partes e avalia/interpola polinômios. O expoente tende a 1 quando k cresce, mas as constantes explodem.

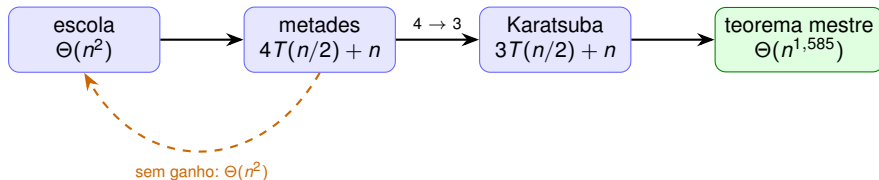
Conjectura-se que $n \log n$ é ótimo: **ninguém provou**. Kolmogorov errou o limite, e o limite certo continua em aberto.

Para aprofundar: Knuth (1997), §4.3.3 [9]; Schönhage e Strassen (1971) [12]; Harvey e van der Hoeven (2021) [5].

O que aprendemos

- Implementações reais para a recursão num ponto de corte e usam a escola dali para baixo. O Python faz exatamente isso no seu `int`.
- A conjectura $\Omega(n^2)$ de Kolmogorov caiu em uma semana: cota inferior exige prova.
- Karatsuba abriu uma família (Toom-Cook, FFT) que chegou a $O(n \log n)$ em 2019. O limite inferior continua em aberto.

Para aprofundar: Karatsuba (1995) [7]; Harvey e van der Hoeven (2021) [5].



O roteiro: escrever a recorrência, ver que as folhas dominam e **cortar um subproblema**, aceitando mais somas. Cortar um subproblema muda o expoente.

Próxima aula: multiplicação de matrizes. Por blocos são 8 produtos e $\Theta(n^3)$; Strassen faz com 7, usando a mesma ideia.

Para aprofundar: Levitin (2012), §5.4 [10].



Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
Introduction to Algorithms.
MIT Press, 3 edition, 2009.



Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
Algoritmos: Teoria e Prática.
Elsevier, Rio de Janeiro, 3 edition, 2012.
Tradução da 3. ed. norte-americana. Seção 4.5.



Sanjoy Dasgupta, Christos H. Papadimitriou, and Umesh V. Vazirani.
Algorithms.
McGraw-Hill, 2008.
Seção 2.1: Multiplication (o truque de Gauss).



Torbjörn Granlund and the GMP development team.
GNU MP: The GNU multiple precision arithmetic library.
Manual, seção “Multiplication Algorithms”.
Acesso em: out. 2026.



David Harvey and Joris van der Hoeven.
Integer multiplication in time $O(n \log n)$.
Annals of Mathematics, 193(2):563–617, 2021.



Anatolii Karatsuba and Yuri Ofman.

Multiplication of multidigit numbers on automata.

Soviet Physics Doklady, 7:595–596, 1963.

Tradução do original russo em *Doklady Akademii Nauk SSSR*, v. 145, n. 2, p. 293–294, 1962.



Anatolii A. Karatsuba.

The complexity of computations.

Proceedings of the Steklov Institute of Mathematics, 211:169–183, 1995.

Relato histórico do seminário de Kolmogorov (1960).



Jon Kleinberg and Éva Tardos.

Algorithm Design.

Pearson/Addison-Wesley, 2006.

Capítulo 5, em especial as seções 5.2 (recorrências) e 5.4 (par mais próximo).



Donald E. Knuth.

The Art of Computer Programming, Volume 2: Seminumerical Algorithms.

Addison-Wesley, 3 edition, 1997.

Seção 4.3.3: How fast can we multiply?



Anany Levitin.

Introduction to the Design and Analysis of Algorithms.
Pearson, 3 edition, 2012.



Python Software Foundation.

CPython: `Objects/longobject.c`.
Código-fonte, constante `KARATSUBA_CUTOFF`.
Acesso em: out. 2026.



Arnold Schönhage and Volker Strassen.

Schnelle Multiplikation großer Zahlen.
Computing, 7(3–4):281–292, 1971.